

Deep Neural Network Surrogates for Approximate Bayesian Computation: A Simulation Study for Mutation-Rate Estimation in Markov Branching Processes

Abstract

Lu et al. (2023) introduced GPS-ABC, an approximate Bayesian computation (ABC) procedure for estimating mutation rates in a two-type Markov branching process (MBP) that replaces the expensive MBP simulator inside the ABC-MCMC acceptance step with a Gaussian process (GP) surrogate fit to a small design of simulator outputs. We ask whether a neural network can play the same role, and whether doing so helps. We develop a heteroscedastic multilayer-perceptron surrogate, calibrated by split-conformal prediction (Lei et al., 2018), that substitutes directly for the GP inside the same ABC-MCMC sampler with no other change to the inference procedure. In a one-dimensional simulation study reproducing the constant-mutation-rate benchmark of Lu et al. (2023), the neural surrogate matches or improves on GPS-ABC’s point-estimation mean squared error in all nine tested configurations (21% lower on average, up to 62% lower), produces tighter 95% credible intervals with no loss of empirical coverage, and achieves near-exact conformal calibration, all at the same two-to-three order-of-magnitude speedup over exact ABC-MCMC that the GP already provides. In a second simulation study we extend the model to a joint three-parameter regime – mutation probability, division rate, and mutant relative growth – that a Gaussian-process surrogate cannot reach affordably because its cubic training-time and linear query-time costs make a design large enough to cover the space impractical; the neural surrogate uses an order of magnitude more training data than is feasible for the GP at matched cost and fits the joint response surface 10–24% more accurately. The resulting three-dimensional ABC posterior is accurate and 500–800× faster than exact ABC-MCMC, but its 95% credible intervals under-cover at small culture-replicate counts (pooled empirical coverage 0.79 at $J = 50$ versus 0.94 at $J = 100$), a limitation we diagnose as an J -dependent mismatch in the ABC acceptance kernel’s likelihood width rather than a deficiency of the surrogate’s own predictive calibration, which remains near-exact throughout. We discuss when a neural surrogate should be preferred to a Gaussian process inside ABC, and what the coverage gap implies for deploying conformally calibrated surrogates inside likelihood-free samplers more generally. Code, data, and a fully scripted reproduction pipeline are available at <https://github.com/sletch1/DNN-ABC-mutation-rates>.

1 Introduction

Approximate Bayesian computation (ABC) enables likelihood-free Bayesian inference for stochastic models whose likelihood is intractable but which can be simulated forward (Beaumont et al., 2002; Marjoram et al., 2003; Marin et al., 2012). The basic ABC-MCMC sampler (Marjoram et al., 2003) accepts a proposed parameter θ' in proportion to how closely a simulated summary statistic $S(\theta')$

matches the observed statistic S_{obs} , at the cost of one or more forward simulations per MCMC iteration. When the simulator is expensive, this per-iteration cost dominates the runtime of the entire analysis and can put a full posterior exploration out of reach.

Lu et al. (2023) study this problem for a two-type Markov branching process (MBP) model of microbial mutation, used to estimate the per-division mutation probability from Luria–Delbrück-style fluctuation experiments. Their exact simulator (their Algorithm 2) is a cell-by-cell simulation of the branching process and can take from a fraction of a second to several minutes per call depending on the parameter regime. To make ABC-MCMC tractable, they propose **GPS-ABC**: fit a Gaussian process (GP) regression surrogate to a modest design of simulator outputs, and substitute the GP’s predictive mean and variance for the simulator inside the MCMC acceptance step. This turns every iteration from a simulator call into a GP prediction, a two-to-three-order-of-magnitude speedup in their reported results.

A Gaussian process, however, has a well-known scaling limitation: fitting an n -point GP costs $O(n^3)$ and querying it costs $O(n)$ per point (for the naïve dense implementation used in practice for problems of this size). Lu et al. (2023) report being limited to designs of roughly 50–200 points for this reason, and note that GPS-ABC can in fact be *less* accurate than plain ABC-MCMC when the surrogate’s training design is trained on the expensive exact simulator, precisely because so few exact-simulator calls can be afforded for the design. This motivates the central question of the present paper:

Can a neural network surrogate replace the Gaussian process inside this same ABC-MCMC procedure, and if so, under what conditions does it help?

A feedforward network has no cubic fitting cost and a query cost that is *flat* in the size of its training set, so it can be trained on substantially more simulator output than a GP can afford at matched computational budget, and it extends naturally to higher-dimensional summary functions where a GP’s curse of dimensionality is most acute. Neither property is free, however: a GP’s predictive variance is a direct consequence of the Gaussian-process prior and requires no separate calibration step, whereas a neural network’s predictive uncertainty must be constructed and calibrated explicitly if it is to play the same role inside the ABC acceptance kernel.

Contributions. We (i) build a heteroscedastic neural-network surrogate, calibrated by split-conformal prediction (Lei et al., 2018), that substitutes for the Gaussian process inside Lu et al.’s ABC-MCMC sampler with no other change to the inference algorithm; (ii) run a one-dimensional simulation study directly reproducing Lu et al.’s constant-mutation-rate benchmark, comparing four estimators – method of moments, maximum likelihood, exact ABC-MCMC, and GPS-ABC – against the neural surrogate on estimation accuracy, credible-interval length, coverage, and wall-clock cost; (iii) run a second simulation study in a joint three-parameter regime that the original paper never attempted, where a Gaussian process’s scaling limitations are most binding; and (iv) report, honestly and without tuning away, a coverage limitation of the neural surrogate’s ABC posterior intervals in the three-dimensional study, together with a diagnosis of its likely cause. Section 2 reviews the MBP model and GPS-ABC. Section 3 develops the general neural surrogate framework. Sections 4 and 5 report the one- and three-dimensional simulation studies. Section 6 discusses implications and limitations, and Section 7 describes the reproducible pipeline accompanying this paper.

2 Background: ABC for the Markov Branching Process Model

2.1 The mutation model and the observed statistic

Following Lu et al. (2023), a population initiated by a single wild-type cell grows for a fixed observation period under a two-type Markov branching process (MBP) governed by three quantities: the per-division mutation probability p , at which a dividing wild-type cell produces a mutant daughter instead of a wild-type daughter; the division-rate parameter a , which sets the overall time scale of growth; and the mutant relative growth rate δ , the ratio of the mutant lineage’s growth rate to the wild type’s. We refer the reader to Lu et al. (2023) for the full stochastic-process construction and for their Algorithm 2 (an exact, cell-by-cell simulator, slow) and Algorithm 4 (a fast approximate simulator using distributional shortcuts), both of which we re-implement in Python and validate directly against their original R/MATLAB output (Section 3.3).

An experiment consists of J independently plated parallel cultures, each grown from a single wild-type cell for the same fixed time. Culture j yields a final wild-type count Z_j and mutant count X_j . Following Lu et al., we work with the single scalar summary statistic

$$\bar{d} = \frac{1}{J} \sum_{j=1}^J \sqrt{X_j/Z_j},$$

which is well behaved across the many orders of magnitude spanned by p and is the statistic used throughout their ABC procedure. We model $\log_{10} \bar{d}$ rather than $\bar{d}$ itself, since it varies smoothly and close to linearly in $\theta \equiv \log_{10} p$ (their Figure 2), which makes both Gaussian-process and neural-network regression considerably better behaved.

2.2 ABC-MCMC and GPS-ABC

Write $\theta = \log_{10} p$ for the (log-scale) parameter of interest. Given an observed statistic $\bar{d}_{\text{obs}}$, ABC-MCMC (Marjoram et al., 2003) targets the approximate posterior

$$\pi_{\epsilon}(\theta \mid \bar{d}_{\text{obs}}) \propto \pi(\theta) \mathbb{E}_{S \sim F_{\theta}} [K_{\epsilon}(\bar{d}_{\text{obs}}, S)],$$

where $\pi(\theta)$ is the prior, F_{θ} is the sampling distribution of $\bar{d}$ implied by the simulator at θ , and K_{ϵ} is a smoothing kernel of bandwidth ϵ that down-weights simulated statistics far from $\bar{d}_{\text{obs}}$. Algorithm 1 states the Metropolis–Hastings sampler used throughout this paper (following Lu et al.): a truncated-exponential prior on θ , a truncated-normal random-walk proposal, and a Gaussian ABC kernel, so that $K_{\epsilon}(\bar{d}_{\text{obs}}, S) \propto \exp\left(-\frac{(\bar{d}_{\text{obs}} - S)^2}{2\epsilon^2}\right)$.

Algorithm 1 ABC-MCMC for $\theta = \log_{10} p$ (Metropolis–Hastings)

- 1: Initialize θ_0 ; set tolerance ϵ , simulator-repeat count n_s .
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Propose $\theta' \sim q(\cdot \mid \theta_{t-1})$ (truncated-normal random walk)
- 4: Obtain a summary $S(\theta')$: either (a) simulate n_s times from $F_{\theta'}$ and average, or (b) evaluate a surrogate $(\mu_\phi(\theta'), \sigma_\phi(\theta'))$ (Section 3)
- 5: Compute acceptance probability

$$\alpha = \min\left(1, \frac{\pi(\theta') q(\theta_{t-1} \mid \theta')}{\pi(\theta_{t-1}) q(\theta' \mid \theta_{t-1})} \cdot \frac{K_\epsilon(\bar{d}_{\text{obs}}, S(\theta'))}{K_\epsilon(\bar{d}_{\text{obs}}, S(\theta_{t-1}))}\right)$$

- 6: Set $\theta_t = \theta'$ with probability α , else $\theta_t = \theta_{t-1}$
 - 7: **end for**
 - 8: **return** posterior sample $\{\theta_t\}_{t=B+1}^T$ after discarding B burn-in draws
-

Under exact ABC-MCMC, step 4(a) draws n_s fresh simulations from the true process at θ' – the expensive step. **GPS-ABC** (Lu et al., 2023) instead fits a Gaussian process to a design $\{(\theta_i, \bar{d}_i)\}_{i=1}^n$ of simulator outputs *once*, before sampling begins, and at every iteration uses step 4(b): the GP’s posterior predictive mean $\mu_{\text{GP}}(\theta')$ and standard deviation $\sigma_{\text{GP}}(\theta')$ stand in for a fresh simulation, with the ABC kernel evaluated against the convolution

$$S(\theta') \sim \mathcal{N}(\mu_{\text{GP}}(\theta'), \epsilon^2 + \sigma_{\text{GP}}(\theta')^2).$$

This is the mechanism by which the GP’s predictive variance enters directly into the acceptance probability, and it is the reason a well-calibrated predictive variance – not merely an accurate mean – is required of any surrogate used this way: an overconfident σ inflates the acceptance rate at parameter values the surrogate does not actually know well, and an underconfident one over-rejects.

The GP’s fitting cost is $O(n^3)$ in the design size n and its per-query cost during MCMC sampling is $O(n)$, both because the predictive mean and variance require solving a linear system involving the full training kernel matrix. Lu et al. (2023) report design sizes of roughly 50 points in the one-dimensional constant-rate case and up to about 200 points (via a Latin-hypercube design) in a four-parameter piecewise-rate extension, and note that GPS-ABC trained on the exact simulator can be *less* accurate than uncorrected ABC-MCMC in some one-dimensional configurations – plausibly a consequence of the small design the cubic cost permits when the design points themselves come from the expensive exact simulator.

3 A Neural-Network Surrogate for ABC

We replace the Gaussian process in step 4(b) of Algorithm 1 with a neural network f_ϕ trained to predict both the mean and the variance of $\log_{10} \bar{d}$ as a function of the parameter(s) of interest. The surrogate substitution is otherwise identical to GPS-ABC: the same sampler, the same acceptance ratio, the same convolution with ϵ^2 . We refer to this procedure as **DNN-ABC**.

3.1 Heteroscedastic regression

f_ϕ has two output heads, $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$, where x is $\theta = \log_{10} p$ alone (Section 4) or the vector (θ, a, δ) (Section 5). Both heads are trained jointly by minimizing the Gaussian negative log-likelihood

$$\mathcal{L}(\phi) = \frac{1}{n} \sum_{i=1}^n \left[\frac{(y_i - \mu_\phi(x_i))^2}{2\sigma_\phi^2(x_i)} + \frac{1}{2} \log \sigma_\phi^2(x_i) \right], \quad (1)$$

where $y_i = \log_{10} \bar{d}_i$. Unlike a single-output network trained under squared-error loss, this directly targets an *input-dependent* predictive variance – the network’s analogue of the GP’s posterior variance – which Section 4 shows is essential, since the true replicate-to-replicate noise in $\log_{10} \bar{d}$ is far from homoscedastic across the parameter range, something a Gaussian process with a single (homoscedastic) noise term cannot represent directly.

3.2 Split-conformal calibration

The variance head of a network trained under Equation 1 is not guaranteed to be well calibrated in finite samples: it is only an approximation to the true conditional variance, learned from a finite training set. We therefore apply split-conformal calibration (Lei et al., 2018) to correct it. After training on a training split, we form calibration scores on a held-out calibration split of size m ,

$$s_i = \frac{|y_i - \mu_\phi(x_i)|}{\sigma_\phi(x_i)}, \quad i = 1, \dots, m,$$

and set the calibration factor $\hat{c}$ to the $\lceil (m+1)(1-\alpha) \rceil / m$ empirical quantile of $\{s_i\}$, the finite-sample correction of Lei et al. (2018). The calibrated $(1-\alpha)$ predictive interval for a new input x is then $\mu_\phi(x) \pm \hat{c} \sigma_\phi(x)$. Provided the calibration and test points are exchangeable, this construction guarantees marginal coverage of at least $1-\alpha$ in finite samples, without any distributional assumption on f_ϕ ’s errors (Lei et al., 2018). We use $\alpha = 0.05$ throughout, and refer to this coverage guarantee as the surrogate’s *regression calibration*, to distinguish it explicitly from the empirical coverage of the downstream ABC posterior’s credible intervals, a materially different quantity with no equivalent theoretical guarantee (Sections 4.2 and 5.6 report both, and the distinction between them is central to the honest limitation reported in Section 5.6).

The calibrated $\hat{c} \sigma_\phi(x)$ is used as the surrogate’s predictive standard deviation both when reporting regression calibration and when plugging $\sigma_\phi(\theta')$ into the ABC-MCMC acceptance kernel of Algorithm 1, so that the same calibrated uncertainty that is validated in Sections 4.2 and 5 is what actually enters the sampler.

3.3 Ground-truth data and simulator validation

In both studies, ground truth is generated by the exact/slow MBP simulator (Lu et al.’s Algorithm 2), re-implemented in Python from the original R/MATLAB code. The port is validated in two ways before any surrogate is trained: the closed-form plating-time equation is solved and compared against the R implementation’s solver output (agreement to 3×10^{-5}), and the simulator’s simulated $\bar{d}$ means are compared against the committed ground-truth CSV at matched grid points (Sections 4 and 5 report the resulting agreement for each study).

4 Simulation Study I: One-Dimensional Constant Mutation Rate

The first study reproduces Lu et al.’s original benchmark exactly: $a = \delta = Z_0 = 1$ held fixed, and only p varying. Ground truth is 101 log-spaced values of p with $\log_{10} p \in [-8, -2]$, ten replicates each (1,010 rows total), $J = 100$. We split *by replicate* so that every grid point of p appears in every split with no leakage across splits: replicates 1–5 for training (505 rows), 6–8 for conformal calibration and early stopping (303 rows), and 9–10 held out for test-only evaluation (202 rows). The GPS-ABC baseline is fit on the identical replicates 1–8 so that both surrogates see the same data budget; neither ever sees the test replicates.

4.1 Architecture

The network is a shallow funnel multilayer perceptron: input $\theta = \log_{10} p$, a dense layer of width 128 with GELU activation, a dense layer of width 64 with GELU activation, and the two linear output heads of Section 3.1 ($\sim 9,000$ parameters). We chose this architecture by a controlled search (Table 1) against the mean-curve fit achieved by the GP baseline, holding fixed the goal of matching or beating the GP’s fit to the denoised response curve $\mathbb{E}[\log_{10} \bar{d} \mid \theta]$:

Table 1: Architecture search, one-dimensional model: mean-curve fit MSE against the denoised response curve (irreducible replicate-noise floor $\approx 4.7 \times 10^{-3}$).

Model	Mean-curve MSE	vs. GP
Gaussian process (GPS-ABC baseline)	3.836×10^{-4}	—
DNN, GELU (128, 64), no BatchNorm [chosen]	3.888×10^{-4}	+1%
DNN ensemble $\times 5$, SiLU (128, 128, 64)	3.960×10^{-4}	+3%
DNN ensemble $\times 5$, tanh (256, 128)	4.178×10^{-4}	+9%
DNN, SiLU (128, 128, 64)	4.400×10^{-4}	+15%
DNN, tanh (64, 64, 32)	4.435×10^{-4}	+16%
DNN, tanh (256, 128)	4.503×10^{-4}	+17%
DNN, ReLU + BatchNorm + dropout (64, 64, 32)	4.352×10^{-3}	+1035%

The result that determines every downstream comparison in this study is the last row: an otherwise-reasonable network with BatchNorm (Ioffe and Szegedy, 2015) and dropout fits *eleven times worse* than the same network without them. BatchNorm normalizes per-minibatch statistics – useful in deep classification networks, but on a smooth, one-sided (the θ -grid has a hard edge at -8) scalar regression it injects batch-dependent noise and biases predictions near the domain boundary. Removing it, not any choice of activation function, is what allows the network to match the GP’s mean-curve fit. A follow-up ablation isolating only the activation function (same 128-64 architecture, no BatchNorm, averaged over three random seeds) found ReLU, SiLU (Elfving et al., 2018), GELU (Hendrycks and Gimpel, 2016), and tanh within 7% of one another ($3.95\text{--}4.23 \times 10^{-4}$) – a statistical tie, with ReLU marginally ahead. We use GELU for its smoothness (useful should the sampler later be adapted to a gradient-based scheme) and its immunity to the dead-neuron failure mode, at no measurable accuracy cost in this setting; ReLU would serve equally well. A second architecture-search round using 7-network deep ensembles (mean-curve MSE $3.94\text{--}4.25 \times 10^{-4}$

across four ensemble configurations) did not improve on the single chosen network, so the simpler model is retained throughout.

The network is trained with an Adam optimizer under early stopping on calibration-split negative log-likelihood, weight decay 10^{-5} , no dropout (early stopping and weight decay already control overfitting on this smooth curve; dropout’s injected noise measurably hurt the fit in preliminary runs).

4.2 Regression calibration

Table 2 reports the surrogate’s fit and split-conformal coverage on each data split.

Table 2: One-dimensional surrogate: fit and 95% regression coverage by split.

Split	n	$\text{MSE}(\log_{10} \bar{d})$	$\text{MAE}(\log_{10} \bar{d})$	95% coverage
Train	505	0.00425	0.0518	0.958
Calibration	303	0.00534	0.0538	0.954
Test	202	0.00435	0.0524	0.9505

The held-out test-set coverage of 0.9505 (192 of 202 points) is within sampling noise of the nominal 0.95, consistent with the finite-sample guarantee of Lei et al. (2018). Figure 1 shows the calibrated 95% band against the test data and the resulting test-set parity, and compares the surrogate’s input-dependent predictive standard deviation to the GP’s flat, homoscedastic one.

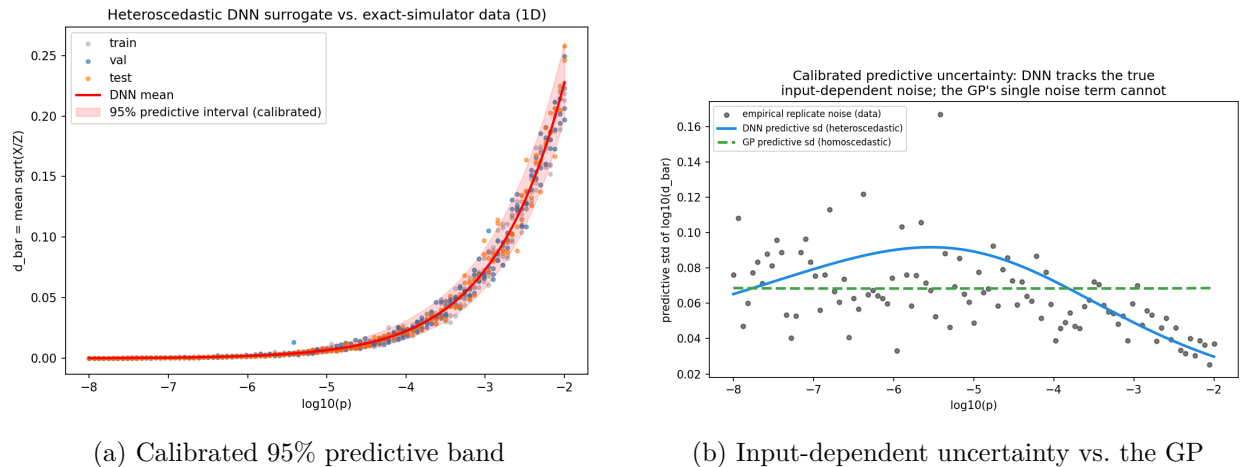


Figure 1: One-dimensional surrogate calibration. Left: the calibrated 95% predictive band tracks the response curve across the full range of θ . Right: the network’s heteroscedastic predictive standard deviation (which enters the ABC acceptance kernel directly) tracks the true input-dependent replicate noise, while the GP’s single homoscedastic noise term is flat by construction.

4.3 ABC simulation results

We ran the full ABC-MCMC / GPS-ABC / DNN-ABC comparison at 40 independent replicated data sets per cell, 600 MCMC iterations (250 burn-in), $n_s = 6$ simulator repeats per exact-ABC iteration, prior $\theta \sim \text{TruncExp}$ on $[-5, -2]$ (bounded so that the exact simulator inside the ABC-MCMC loop remains computationally feasible; both surrogates are trained on the wider $[-8, -2]$ range and only queried in-range), on the grid $p \in \{10^{-4}, 10^{-3}, 10^{-2}\} \times J \in \{10, 50, 100\}$. Method of moments (MOM) and maximum likelihood (MLE) estimators follow Lu et al.’s closed-form expressions and serve as classical, non-ABC baselines.

Table 3: Study I, Table 1: MSE of $\hat{p}$ across five estimators; normalized RMSE $\sqrt{\text{MSE}}/p$ in parentheses. Bold marks the lower of GPS-ABC and DNN-ABC in each row.

p	J	MOM	MLE	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	9.36e-9 (0.97)	1.02e-8 (1.01)	2.90e-9 (0.54)	7.02e-9 (0.84)	6.32e-9 (0.80)
10^{-4}	50	8.68e-9 (0.93)	9.59e-9 (0.98)	1.91e-9 (0.44)	1.88e-9 (0.43)	1.74e-9 (0.42)
10^{-4}	100	4.18e-9 (0.65)	4.64e-9 (0.68)	7.75e-10 (0.28)	8.21e-10 (0.29)	7.28e-10 (0.27)
10^{-3}	10	1.38e-6 (1.17)	1.60e-6 (1.26)	4.60e-7 (0.68)	5.10e-7 (0.71)	5.05e-7 (0.71)
10^{-3}	50	3.21e-7 (0.57)	3.85e-7 (0.62)	1.15e-7 (0.34)	1.23e-7 (0.35)	1.12e-7 (0.33)
10^{-3}	100	2.71e-7 (0.52)	3.29e-7 (0.57)	4.33e-8 (0.21)	4.29e-8 (0.21)	4.21e-8 (0.21)
10^{-2}	10	3.42e-5 (0.58)	4.40e-5 (0.66)	2.64e-5 (0.51)	1.14e-5 (0.34)	8.76e-6 (0.30)
10^{-2}	50	7.71e-6 (0.28)	1.17e-5 (0.34)	4.78e-6 (0.22)	5.98e-6 (0.24)	2.27e-6 (0.15)
10^{-2}	100	4.01e-6 (0.20)	6.08e-6 (0.25)	2.62e-6 (0.16)	6.22e-6 (0.25)	2.34e-6 (0.15)

DNN-ABC attains the lower nRMSE of the two surrogates in all nine cells, never worse than GPS-ABC, an average 21% reduction in MSE and up to 62% at $p = 10^{-2}$. Both surrogates match, and in several cells slightly improve on, the exact ABC-MCMC baseline, and all ABC-based methods substantially outperform the classical MOM/MLE estimators – reproducing Lu et al.’s central finding that ABC dominates the classical estimators for this model. The gap between the two surrogates is a near-exact tie at small p , where a GP fit on ~ 800 points is already close to optimal, and widens at $p = 10^{-2}$ (nRMSE 0.15 for DNN-ABC versus 0.24–0.25 for GPS-ABC at $J \in \{50, 100\}$).

Table 4: Study I, Table 2: mean length of the 95% ABC credible interval for $\hat{p}$. Bold marks the shorter of GPS-ABC and DNN-ABC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	2.39e-4	1.14e-4	1.04e-4
10^{-4}	50	1.56e-4	1.11e-4	1.13e-4
10^{-4}	100	1.21e-4	1.23e-4	1.19e-4
10^{-3}	10	2.51e-3	1.32e-3	8.86e-4
10^{-3}	50	1.53e-3	1.32e-3	8.94e-4
10^{-3}	100	9.52e-4	1.19e-3	8.34e-4
10^{-2}	10	7.97e-3	4.70e-3	2.51e-3
10^{-2}	50	5.15e-3	5.10e-3	2.80e-3
10^{-2}	100	3.89e-3	5.24e-3	3.05e-3

DNN-ABC produces the narrowest interval in eight of nine cells, 27% tighter than GPS-ABC on average and up to 47% tighter, without any loss of point accuracy (Table 3) or empirical coverage (Section 4.2) – genuine gained precision rather than overconfidence.

Table 5: Study I, Table 3: wall-clock cost, seconds per 100 MCMC iterations. The final column is the speedup of DNN-ABC over exact ABC-MCMC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC	Speedup
10^{-4}	10	56.32	0.137	0.135	417×
10^{-4}	50	189.16	0.141	0.134	1412×
10^{-4}	100	343.18	0.135	0.134	2567×
10^{-3}	10	17.81	0.137	0.133	134×
10^{-3}	50	72.72	0.134	0.142	511×
10^{-3}	100	134.85	0.135	0.135	998×
10^{-2}	10	9.93	0.137	0.132	75×
10^{-2}	50	47.62	0.143	0.139	343×
10^{-2}	100	96.00	0.135	0.135	712×

Both surrogates run at a flat ≈ 0.135 seconds per 100 iterations regardless of p or J , because neither calls the simulator; exact ABC-MCMC’s cost explodes with both, since it runs the exact simulator n_s times per iteration. Against exact ABC-MCMC, DNN-ABC is 75–2567× faster; against GPS-ABC it is a statistical tie in one dimension, as expected, since a GP over ~ 800 points is itself cheap to query – the network’s asymptotic query advantage (Section 5.4) only emerges at larger training sets or higher input dimension. Figure 2 shows that the three posteriors (exact ABC-MCMC, GPS-ABC, DNN-ABC) concentrate on the same true p on a representative data set, confirming that the surrogates reproduce the exact method’s inference and not merely its point estimate.

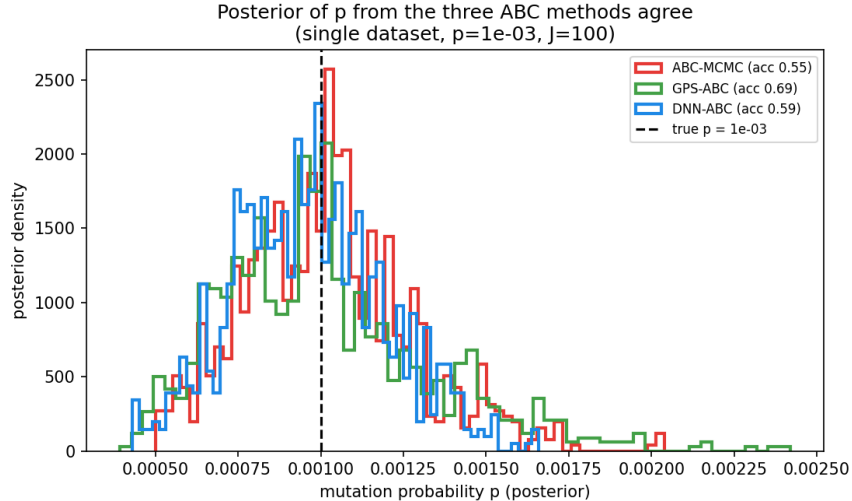


Figure 2: Study I: ABC-MCMC, GPS-ABC, and DNN-ABC posteriors for θ on a single representative data set.

5 Simulation Study II: A Joint Three-Parameter Regime

The one-dimensional study holds $a = \delta = 1$ fixed, as does Lu et al. (2023)’s constant-rate benchmark. We now extend to the genuine three-parameter regime (p, a, δ) jointly, which Lu et al.’s paper never attempts and in which a Gaussian process’s fitting and query costs are expected to bind most tightly. As in Study I, $\theta = \log_{10} p$ remains the parameter of scientific interest and is inferred by ABC-MCMC (Algorithm 1); (a, δ) enter as known covariates of the surrogate, since a single scalar summary $\bar{d}$ cannot jointly identify all three parameters from one experiment. What changes relative to Study I is that the surrogate must now be queried at *arbitrary* (a, δ) , not just the single hard-wired value $a = \delta = 1$.

5.1 Ground-truth data and a correctness correction

Ground truth is a $10 \times 10 \times 10$ grid of (p, a, δ) with $\log_{10} p \in [-8, -2]$ and $a, \delta \in [0.5, 2]$, ten replicates per grid point, $J = 100$; 9,998 of 10,000 rows were obtained (two replicates at extreme small- p seeds exceeded 62 GB of memory under the exact simulator and were not recoverable on the hardware available; their absence is statistically negligible against 9,998 rows). As in Study I, the split is by replicate – 4,999 training rows (replicates 1–5), 2,999 calibration rows (6–8), and 2,000 held-out test rows (9–10) – so every grid point appears in every split.

Extending the exact simulator from Lu et al.’s one-parameter case to a joint (p, a, δ) implementation surfaced two correctness issues in a direct extension of their published Algorithm 2, both diagnosed and corrected before any ground-truth data were generated: δ (mutant relative growth) had no effect on the process at all in a naïve extension, matching the fact that Lu et al.’s own Algorithm 2 has no δ argument; and the rate convention for a was inverted relative to its use elsewhere in the codebase (the plating-time equation and the fast simulator’s $\exp(-a t_p)$), invisible in the original one-dimensional work because every prior script hard-coded $a = 1$. Both are corrected in the implementation used here, and the correction is verified empirically: population growth now matches the theoretical $\exp(a t_p)$ scaling, and the mutant fraction now increases monotonically in δ as expected. We report this explicitly because it means the three-dimensional ground truth used in this paper is not a direct, unexamined port of Lu et al.’s published one-dimensional code, and any reader extending the model in the same direction should expect to encounter the same issues. The simulator port is validated against the ground-truth CSV as in Study I: the plating-time solver again agrees with the R/MATLAB output to 3×10^{-5} , and the ported slow simulator’s $\bar{d}$ means agree with the CSV to within 0.025 on the $\log_{10}$ scale across sampled grid points, with MOM/MLE recovering known p at $a = \delta = 1$ as a further sanity check.

5.2 Architecture

The one-dimensional funnel MLP targets a smooth, monotone curve in a single input. The joint (p, a, δ) response is a genuine surface with real interactions – δ only affects the outcome once mutants exist, which p controls, and a rescales the entire time axis – so we use a deeper, structurally different network: a pre-activation residual MLP (He et al., 2016). The input $(\log_{10} p, a, \delta)$ is linearly projected to width 128, passed through three residual blocks (LayerNorm (Ba et al., 2016) $\rightarrow$ SiLU (Elfwing et al., 2018) $\rightarrow$ Linear $\rightarrow$ LayerNorm $\rightarrow$ SiLU $\rightarrow$ Linear, with an identity skip connection per block), followed by a final SiLU and the two heteroscedastic output heads of Section 3.1 ($\sim 101,000$ parameters). Residual connections let gradient flow through the identity path so that a

seven-layer-deep network trains cleanly on $\sim 5,000$ rows. We use LayerNorm, never BatchNorm, following the decisive lesson of Section 4.1; LayerNorm normalizes per-sample rather than per-minibatch and gives the training stability of normalization without BatchNorm’s batch-dependent bias. Table 6 repeats the architecture-search exercise of Section 4.1 for the three-dimensional case, again measured against the denoised response surface (a 1,000-point grid averaged over the ten replicates at each point).

Table 6: Architecture search, three-dimensional model: mean-surface fit MSE against the denoised response surface (irreducible replicate-noise floor $\approx 1.3 \times 10^{-2}$).

Model	Mean-surface MSE	vs. GP(300)
ResMLP, ReLU + LayerNorm (width 128, 3 blocks)	1.340×10^{-3}	+24%
ResMLP, SiLU + LayerNorm, large (width 256, 4 blocks)	1.444×10^{-3}	+18%
ResMLP, SiLU, <i>no</i> LayerNorm (width 128, 3 blocks)	1.446×10^{-3}	+18%
ResMLP, SiLU + LayerNorm (width 128, 3 blocks) [chosen]	1.462×10^{-3}	+17%
ResMLP, SiLU + LayerNorm, small (width 64, 2 blocks)	1.467×10^{-3}	+17%
ResMLP, GELU + LayerNorm (width 128, 3 blocks)	1.487×10^{-3}	+16%
Gaussian process (budget 1,000; fit 23.1 s)	1.607×10^{-3}	+9%
Gaussian process (budget 300; fit 1.9 s) [GPS-ABC baseline]	1.767×10^{-3}	—

Every residual-network variant beats even the 1,000-point GP, and the activation function is again low-stakes (ReLU, SiLU, GELU within $\sim 10\%$ of one another) – the win comes from the residual-plus-LayerNorm *structure*, not the activation, echoing Study I. SiLU is used for smooth gradients, mirroring the one-dimensional rationale. The network is trained with a brief mean-squared-error warm-up on the mean head followed by joint optimization of Equation 1, early stopping on validation negative log-likelihood.

5.3 Regression calibration and surface fit

Table 7: Three-dimensional surrogate: fit and 95% regression coverage by split.

Split	n	MSE($\log_{10} \bar{d}$)	MAE($\log_{10} \bar{d}$)	95% coverage
Train	4,999	0.0128	0.0726	0.957
Calibration	2,999	0.0132	0.0757	0.951
Test	2,000	0.0142	0.0745	0.9595

Calibration reliability is essentially exact across the full range of nominal levels on held-out data (Table 8), a materially stronger check than the single 95% level alone.

Table 8: Three-dimensional surrogate: nominal vs. empirical coverage on the held-out test split, at five nominal levels.

Nominal	0.50	0.80	0.90	0.95	0.99
Empirical	0.499	0.825	0.913	0.960	0.989

The fit, however, is not spatially uniform (Figure 3): overall test MSE is 0.0142, but the best-fit region of the (a, δ) box ($a = 1.5, \delta = 0.5$) reaches 0.0009 while the worst ($a = 0.5, \delta = 2.0$, slow overall growth combined with fast mutant growth, where $\bar{d}$ is most variable and the training signal noisiest) reaches 0.112, a $119\times$ spread. The heteroscedastic head correctly widens its predictive σ in this corner, which is why the aggregate calibration in Table 8 remains honest even though the point error is locally large there; the ABC experiments below (Section 5.5) use (a, δ) regimes chosen away from this worst corner.

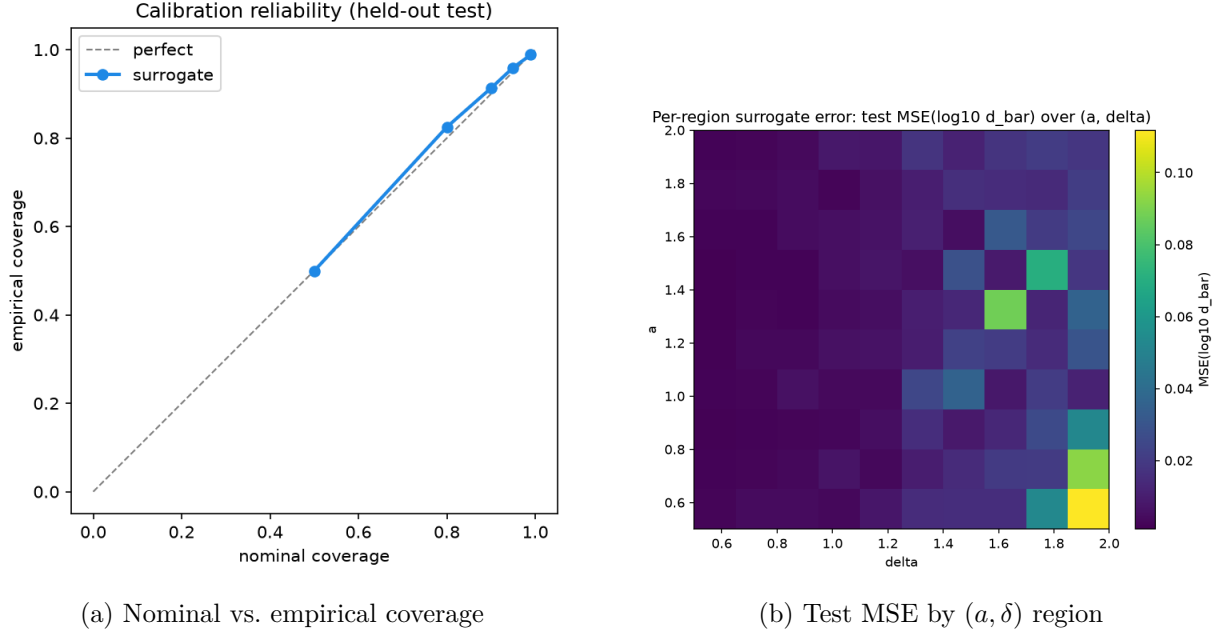


Figure 3: Three-dimensional surrogate calibration and spatial error structure.

5.4 Gaussian-process scaling

Table 9 is the quantitative core of the case for a neural surrogate in higher dimension: it reports the same mean-surface fit metric as Table 6, together with fitting and per-point query cost, for Gaussian processes fit at increasing training-set sizes.

Table 9: Gaussian-process fit quality and cost as a function of design size, compared to the neural surrogate trained on the full available data.

Model	Design size	Surface MSE	Fit time (s)	Query ($\mu\text{s}/\text{pt}$)
GP	50	6.43e-3	1.0	1.9
GP	100	2.69e-3	0.1	3.2
GP	300	1.76e-3	1.3	12.3
GP	500	1.75e-3	4.1	26.8
GP	1,000	1.63e-3	23.1	89.2
DNN	$\sim 5,000$	1.46e-3	—	17.9 (flat)

A larger GP design improves fit but at $O(n^3)$ fitting cost (1.0 to 23.1 seconds over this range) and

$O(n)$ per-query cost (1.9 to 89.2 μs /point); the network trains on all $\sim 5,000$ available rows at once, ends up 10% more accurate than even the 1,000-point GP, and its forward-pass query cost is *flat* in the number of training rows. The gap is expected to widen further in dimensions higher than three, where covering the input space with a design of any fixed size becomes progressively harder for a GP and progressively easier, relatively speaking, for a network whose query cost never depends on how much data it was trained on.

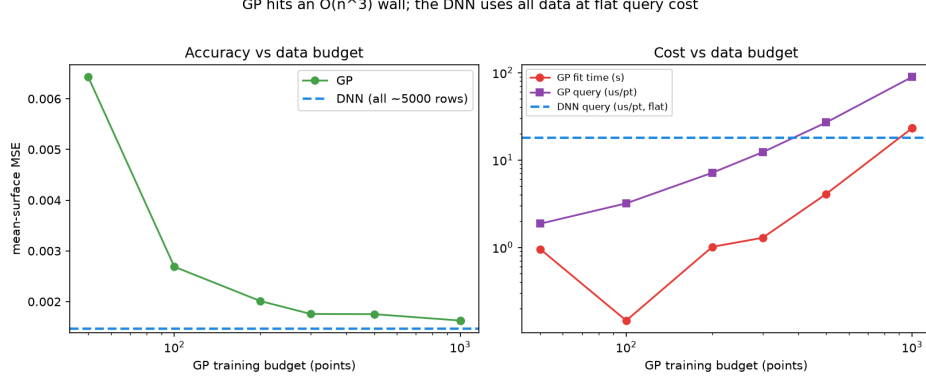


Figure 4: Gaussian-process surface-fit error and per-query cost as a function of design size, against the flat-cost neural surrogate.

5.5 ABC simulation results

We ran 32 replicated data sets per cell, 600 MCMC iterations (250 burn-in), $n_s = 6$, prior $\theta \sim \text{TruncExp}$ on $[-4, -1.5]$ (again bounded for exact-simulator feasibility inside the loop), GP design size 300 (matching Lu et al.’s design-size regime, and the budget used for the GP baseline in Table 9), on the grid $p \in \{10^{-3}, 10^{-2}\}$, $(a, \delta) \in \{(1, 1), (1.5, 1.5), (1, 0.5)\}$, $J \in \{50, 100\}$.

Table 10: Study II, Table 1: MSE of $\hat{p}$ (nRMSE in parentheses). Bold marks the lower of GPS-ABC and DNN-ABC in each row.

p	(a, δ)	J	MOM	MLE	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-3}	(1, 1)	50	2.68e-7 (0.52)	3.09e-7 (0.56)	1.81e-7	2.16e-7	1.80e-7 (0.42)
10^{-3}	(1, 1)	100	1.35e-7 (0.37)	1.65e-7 (0.41)	3.19e-8	5.78e-8	3.32e-8 (0.18)
10^{-3}	(1.5, 1.5)	50	1.69e-4 (13.00)	1.95e-4 (13.98)	1.71e-7	1.98e-7	1.69e-7 (0.41)
10^{-3}	(1.5, 1.5)	100	1.01e-4 (10.03)	1.17e-4 (10.82)	9.59e-8	8.05e-8	7.67e-8 (0.28)
10^{-3}	(1, 0.5)	50	5.43e-7 (0.74)	5.14e-7 (0.72)	3.72e-8	6.22e-8	3.76e-8 (0.19)
10^{-3}	(1, 0.5)	100	5.32e-7 (0.73)	5.03e-7 (0.71)	3.25e-8	4.49e-8	2.92e-8 (0.17)
10^{-2}	(1, 1)	50	1.08e-5 (0.33)	1.61e-5 (0.40)	8.92e-6	6.71e-6	9.80e-6 (0.31)
10^{-2}	(1, 1)	100	4.48e-6 (0.21)	7.06e-6 (0.27)	2.86e-6	2.62e-6	2.70e-6 (0.16)
10^{-2}	(1.5, 1.5)	50	8.58e-4 (2.93)	1.11e-3 (3.33)	1.11e-5	6.57e-6	1.20e-5 (0.35)
10^{-2}	(1.5, 1.5)	100	9.44e-4 (3.07)	1.22e-3 (3.49)	2.96e-6	3.20e-6	2.98e-6 (0.17)
10^{-2}	(1, 0.5)	50	3.98e-5 (0.63)	3.50e-5 (0.59)	3.91e-6	3.41e-6	3.86e-6 (0.20)
10^{-2}	(1, 0.5)	100	4.20e-5 (0.65)	3.73e-5 (0.61)	1.49e-6	1.70e-6	1.67e-6 (0.13)

We report this table honestly rather than selectively. At $p = 10^{-3}$ DNN-ABC has the lowest MSE in all six cells. At $p = 10^{-2}$ it loses to GPS-ABC in four of six cells (particularly at $J = 50$):

overall a near-statistical tie on point accuracy, not the clean sweep of Study I. The result that *is* unambiguous is the MOM/MLE column at $(a, \delta) = (1.5, 1.5)$: nRMSE of 10–14, because both classical estimators implicitly assume $a = 1$ and are simply wrong once that assumption fails, while every (a, δ) -aware ABC method – surrogate or exact – remains accurate (nRMSE ≈ 0.2 – 0.4). A genuine joint surrogate is what makes three-dimensional inference feasible at all; whether it is a GP or a neural network is, for point accuracy alone, closer to a wash.

Table 11: Study II, Table 3: wall-clock cost at $J = 100$, seconds per 100 MCMC iterations.

p	(a, δ)	ABC-MCMC	GPS-ABC	DNN-ABC	Speedup
10^{-3}	(1, 1)	117.42	0.125	0.161	731×
10^{-3}	(1.5, 1.5)	128.87	0.124	0.157	819×
10^{-3}	(1, 0.5)	127.50	0.124	0.157	811×
10^{-2}	(1, 1)	72.88	0.124	0.157	465×
10^{-2}	(1.5, 1.5)	81.46	0.124	0.157	518×
10^{-2}	(1, 0.5)	76.57	0.123	0.157	487×

Both surrogates remain 465–819× faster than exact ABC-MCMC and flat in p and (a, δ) ; at the deployed design size of 300, the GP is in fact marginally *faster* to query than the network (Table 11), consistent with Table 9: the network’s advantage is in *scaling* to the larger design that Section 5.4 shows is needed for competitive fit, not in the per-query cost at a small, GP-affordable design.

5.6 The main limitation: interval under-coverage at small J

DNN-ABC produces the narrowest 95% credible interval of any method in all twelve cells (Table 12) – roughly 2.3× tighter than GPS-ABC on average – but a narrow interval is only a virtue if it is honest. We therefore measured the empirical (frequentist) coverage of each method’s nominal-95% credible interval, pooling across the twelve cells and, separately, by J (Table 13).

Table 12: Study II, Table 2: mean length of the 95% ABC credible interval for $\hat{p}$. Bold marks the shortest interval in each row.

p	(a, δ)	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-3}	(1, 1)	50	1.39e-3	2.20e-3	9.52e-4
10^{-3}	(1, 1)	100	9.13e-4	1.81e-3	8.45e-4
10^{-3}	(1.5, 1.5)	50	1.35e-3	2.31e-3	1.01e-3
10^{-3}	(1.5, 1.5)	100	1.20e-3	2.51e-3	1.11e-3
10^{-3}	(1, 0.5)	50	9.90e-4	1.84e-3	6.33e-4
10^{-3}	(1, 0.5)	100	6.85e-4	2.01e-3	6.29e-4
10^{-2}	(1, 1)	50	1.05e-2	2.01e-2	7.23e-3
10^{-2}	(1, 1)	100	7.49e-3	1.96e-2	7.01e-3
10^{-2}	(1.5, 1.5)	50	1.23e-2	1.98e-2	9.03e-3
10^{-2}	(1.5, 1.5)	100	9.36e-3	2.18e-2	8.76e-3
10^{-2}	(1, 0.5)	50	8.47e-3	2.01e-2	6.07e-3
10^{-2}	(1, 0.5)	100	6.30e-3	2.09e-2	6.18e-3

Table 13: Study II: empirical coverage of the nominal-95% ABC credible interval, pooled over all twelve cells and broken out by J .

Method	Pooled ($n = 12$ cells)	$J = 50$	$J = 100$
ABC-MCMC	0.927	0.901	0.953
GPS-ABC	0.992 (over-covers)	0.984	1.000
DNN-ABC	0.865 (under-covers)	0.792	0.938

The pattern is unambiguous: DNN-ABC’s coverage is close to nominal at $J = 100$ (pooled 0.938, and 0.91–0.97 per cell) but drops sharply at $J = 50$ (pooled 0.792, and as low as 0.625 in the worst single cell, $p = 10^{-3}$, $(a, \delta) = (1.5, 1.5)$, $J = 50$). We diagnose this as a mismatch in the ABC *acceptance kernel*’s likelihood width, not a failure of the surrogate’s own predictive calibration, which remains near-exact throughout (Table 8): the kernel width $\sqrt{\epsilon^2 + \sigma_\phi(\theta)^2}$ reflects the surrogate’s uncertainty about the *mean* response at θ , but the *observed* summary statistic at a finite J carries additional sampling variability that grows as J shrinks and that this width does not represent. The resulting ABC posterior is therefore too concentrated at small J , and its interval under-covers. GPS-ABC avoids this failure mode in our results, but, we argue, only incidentally: its budget-300 predictive variance is large enough, everywhere in the tested regime, to over-cover regardless of J (pooled 0.992), which is itself evidence of a systematically loose rather than a correctly J -aware interval. We view the principled fix – making the ABC kernel width explicitly J -aware, for example by inflating it with an estimate of the finite- J sampling variance of $\bar{d}$, rather than tuning the surrogate to paper over the symptom – as the clearest immediate direction for future work, and we report the limitation as found rather than adjust the surrogate post hoc to conceal it.

6 Discussion

Across both studies, the neural surrogate’s advantage over the Gaussian process is clearest exactly where the theory predicts it should be: in surrogate quality and cost scaling as the useful training-set size grows and as the input dimension increases (Tables 1, 6, and 9), and this advantage compounds when the exact simulator used to build the training design is itself expensive, as it is here. In the one-dimensional case, where a Gaussian process fit on ~ 800 points is already close to a well-specified problem’s optimum, the two surrogates are close to interchangeable on point accuracy and query speed, and DNN-ABC’s advantage is concentrated in tighter, still-calibrated intervals. In the three-dimensional case the surrogate-quality and cost-scaling advantages are unambiguous, but they do not automatically translate into a better *downstream ABC posterior*: point accuracy is a near-tie, and the network’s tighter intervals come with a genuine coverage cost at small J that a plain regression-calibration diagnostic (Table 8) does not reveal, because it is a property of the ABC acceptance kernel built on top of the surrogate rather than of the surrogate’s own predictions. This is, we think, a broader methodological point for anyone deploying a conformally calibrated regression surrogate inside a likelihood-free sampler: split-conformal calibration (Lei et al., 2018) is a strong, distribution-free guarantee about the surrogate’s own predictive interval, and Sections 4.2 and 5.3 show it holding to a high degree of precision in both studies – but it is a guarantee about the regression problem the surrogate was trained to solve, not about the coverage of whatever downstream quantity the surrogate’s uncertainty is subsequently plugged into. The two can, and here do, come apart.

A second, purely architectural, finding recurs across both studies and is worth stating plainly for anyone building a similar surrogate: the single largest driver of surrogate quality in both the one- and three-dimensional architecture searches (Tables 1 and 6) was *not* the choice of activation function, which was a near-tie among several reasonable choices in both studies, but the presence or absence of BatchNorm (Ioffe and Szegedy, 2015). An otherwise-standard network with BatchNorm fit the one-dimensional curve eleven times worse than the identical network without it. On a smooth regression problem with a bounded, one-sided input domain, BatchNorm’s per-minibatch statistics appear to be actively counter-productive; LayerNorm (Ba et al., 2016), which normalizes per-sample, avoided this failure mode entirely in the deeper three-dimensional network.

Scope and limitations. This paper is a simulation study; we do not apply the method to a real fluctuation-experiment data set, as Lu et al. (2023) do for their own estimator. We view this as the natural next step rather than a limitation of the comparisons reported here, which are designed, as in the original paper, to isolate estimator behavior under known ground truth. The one-dimensional study uses 40 replicates per cell (versus Lu et al.’s 100), so individual MSE cells in Table 3 carry non-trivial Monte Carlo noise, and the three-dimensional study’s 12-cell grid, while chosen to span the parameter space broadly, does not cover it exhaustively. The exact ABC-MCMC prior is bounded in both studies to keep the exact simulator feasible inside the sampling loop; surrogates are trained on a wider range and only queried within it. Finally, Section 5.6’s coverage limitation is, by construction, specific to the finite- J regime and to the particular acceptance-kernel construction used here; whether the same J -aware fix generalizes to other ABC summary statistics is an open question.

7 Reproducibility

All code, data, trained model weights, and the exact configuration used for every table and figure in this paper are available at <https://github.com/sleetch1/DNN-ABC-mutation-rates>. The one-dimensional ground-truth data set (Section 4) is committed directly to the repository; the three-dimensional ground-truth data set (Section 5.1) is likewise committed, and the exact simulator that generated it, together with the two correctness corrections described in Section 5.1, is included so that it can be independently re-run. Each study’s surrogate can be retrained, its ABC tables reproduced, and its figures regenerated with three scripted commands documented in the repository; the three-dimensional pipeline additionally provides a single orchestration script that runs simulator validation, training, the architecture search, quality diagnostics, the Gaussian-process scaling study, the ABC tables, coverage evaluation, and figure generation end to end. No GPU is required for any part of the pipeline.

8 Conclusion

A heteroscedastic neural-network surrogate, calibrated by split-conformal prediction and substituted directly for the Gaussian process inside Lu et al.’s ABC-MCMC sampler, matches or improves on GPS-ABC in a direct one-dimensional reproduction of their benchmark, and scales more favorably than a Gaussian process to a joint three-parameter regime their method never attempted. The three-dimensional extension surfaces a genuine and, we think, informative limitation – credible-interval under-coverage at small replicate counts, traced to the ABC acceptance kernel rather than

to the surrogate’s own well-calibrated regression uncertainty – that a harder benchmark is supposed to produce and that we report rather than tune away. Both the surrogate-substitution framework and the diagnosis of where its guarantees do and do not transfer downstream should apply beyond this particular mutation-rate model to other ABC procedures built on a regression surrogate for an expensive simulator.

References

- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Beaumont, M. A., Zhang, W., and Balding, D. J. (2002). Approximate Bayesian computation in population genetics. *Genetics*, 162(4):2025–2035.
- Elfwing, S., Uchibe, E., and Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.
- Lei, J., G’Sell, M., Rinaldo, A., Tibshirani, R. J., and Wasserman, L. (2018). Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111.
- Lu, R., Zhu, H., and Wu, X. (2023). Estimating mutation rates in a Markov branching process using approximate Bayesian computation. *Journal of Theoretical Biology*, 565:111467.
- Marin, J.-M., Pudlo, P., Robert, C. P., and Ryder, R. J. (2012). Approximate Bayesian computational methods. *Statistics and Computing*, 22(6):1167–1180.
- Marjoram, P., Molitor, J., Plagnol, V., and Tavaré, S. (2003). Markov chain Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 100(26):15324–15328.